LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 10 – Nilai Max/Min

NAMA : Adib Hanan Kautsar

NIM : 109082500182

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pencarian nilai ekstrim, yang meliputi pencarian nilai maksimum dan minimum, merupakan salah satu algoritma fundamental dalam pemrograman untuk mengolah sekumpulan data atau array. Algoritma ini bekerja dengan prinsip perbandingan sekuensial, di mana setiap elemen dalam kumpulan data diperiksa satu per satu untuk menentukan nilai yang paling besar atau paling kecil sesuai kriteria yang diinginkan. Ide dasar dari algoritma ini adalah menetapkan elemen pertama dari data sebagai nilai acuan awal (nilai ekstrim sementara), kemudian membandingkannya secara bertahap dengan elemen-elemen berikutnya hingga seluruh data selesai divalidasi.

Dalam implementasinya menggunakan bahasa pemrograman Go, pencarian nilai ekstrim dapat dilakukan baik pada tipe data dasar (seperti integer atau float) maupun pada tipe data terstruktur (struct). Untuk data bertipe struct, pencarian sering kali difokuskan pada salah satu atribut tertentu, misalnya mencari mahasiswa dengan IPK tertinggi atau balita dengan berat badan terkecil. Penting untuk diperhatikan bahwa dalam pencarian ini, selain nilai ekstrim itu sendiri, posisi atau indeks dari data tersebut dalam array sering kali menjadi informasi yang lebih krusial agar seluruh atribut data terkait dapat diakses kembali.

Penggunaan subprogram berupa fungsi dan prosedur sangat disarankan untuk menjaga efisiensi dan keterbacaan kode, terutama pada kasus yang kompleks. Prosedur dapat digunakan untuk menghasilkan dua keluaran sekaligus, seperti nilai minimum dan maksimum secara bersamaan, dengan memanfaatkan parameter pointer agar perubahan nilai di dalam subprogram dapat tersimpan pada variabel di program utama. Sementara itu, fungsi biasanya digunakan untuk mengembalikan satu nilai spesifik, seperti hasil perhitungan rata-rata dari sekumpulan data yang ada di dalam array.

B. Guided

1. [guided1]

```
package main

import "fmt"
 func nilaiTerbesar(array []int) (int,
int) {     var idx int     var
nilaiTerbesar int = array[0]     var
idxDitemukan int = 0     for idx = 1; idx
< len(array); idx++ {         if
array[idx] > nilaiTerbesar {
nilaiTerbesar = array[idx]
```

```
            idxDitemukan = idx
        }     }     return
nilaiTerbesar, idxDitemukan
}  func nilaiTerkecil(array []int) (int, int)
{     var idx int     var nilaiTerkecil int =
array[0]     var idxDitemukan int = 0     for
idx = 1; idx < len(array); idx++ {         if
array[idx] < nilaiTerkecil {
nilaiTerkecil = array[idx]
idxDitemukan = idx
        }     }     return
nilaiTerkecil, idxDitemukan
}  func main() {     var arrAngka [5]int = [5]int{45,
23, 98, 66, 12}
     var nilaiMaks, idxNilaiMaks int = nilaiTerbesar(arrAngka[:])
var nilaiMin, idxNilaiMin int = nilaiTerkecil(arrAngka[:])
     fmt.Printf("Nilai terkecil : %d ditemukan pada index ke-%d",
nilaiMin, idxNilaiMin)     fmt.Println()     fmt.Printf("Nilai
terbesar : %d ditemukan pada index ke-%d", nilaiMaks,
idxNilaiMaks)

}
```

Output :

```
sar\Week 9> cd "Guided"
PS C:\Users\LENOVO\Desktop\109082500182_Adib-Hanan-Kaut
sar\Week 9\Guided> go run soal1.go
Nilai terkecil : 12 ditemukan pada index ke-4
Nilai terbesar : 98 ditemukan pada index ke-2
PS C:\Users\LENOVO\Desktop\109082500182_Adib-Hanan-Kaut
```

Penjelasan :

Program ini terdiri dari fungsi-fungsi kecil untuk menemukan nilai terbesar dan nilai terkecil dalam sebuah slice bilangan bulat.

2. [guided2]

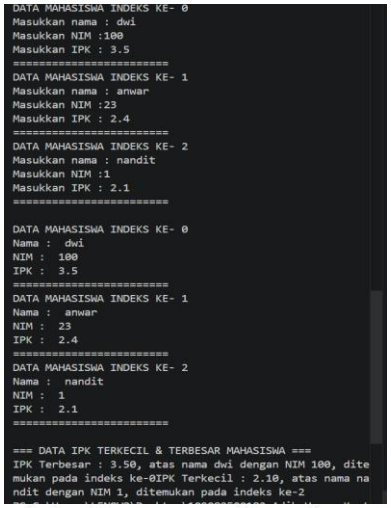
```
package main
```

```
 import "fmt"
 type  mahasiswa  struct  {        nama
string     nim  string
    IPK  float64
}  type arrMahasiswa [3]mahasiswa
 func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {     var
mhsTerbesar mahasiswa = array[0]     var idxDitemukan int = 0
var i int
     for i = 1; i < len(array); i++ {         if
array[i].IPK > mhsTerbesar.IPK {             mhsTerbesar
= array[i]             idxDitemukan = i
        }
    }      return mhsTerbesar, idxDitemukan }  func
IPKTerkecil(array []mahasiswa) (mahasiswa, int) {     var
mhsTerkecil mahasiswa = array[0]     var idxDitemukan int = 0
var i int

    for i = 1; i < len(array); i++ {         if
array[i].IPK < mhsTerkecil.IPK {             mhsTerkecil
= array[i]             idxDitemukan = i
        }     }     return mhsTerkecil, idxDitemukan }  func
main() {     var arrMhs arrMahasiswa     var i, j int     for i =
0; i < len(arrMhs); i++ {         fmt.Println("DATA MAHASISWA
INDEKS KE-", i)
```

```
        fmt.Print("Masukkan nama : ")
fmt.Scan(&arrMhs[i].nama)
fmt.Print("Masukkan NIM :")
fmt.Scan(&arrMhs[i].nim)
fmt.Print("Masukkan IPK : ")
fmt.Scan(&arrMhs[i].IPK)
fmt.Println("========================")
    }      fmt.Println()     for j = 0; j <
len(arrMhs); j++ {         fmt.Println("DATA
MAHASISWA INDEKS KE-", j)
fmt.Println("Nama : ", arrMhs[j].nama)
fmt.Println("NIM : ", arrMhs[j].nim)
fmt.Println("IPK : ", arrMhs[j].IPK)
fmt.Println("========================")
    }
fmt.Println()
     var mhsMaks, mhsMin
mahasiswa     var idxMaks,
idxMin int
     mhsMaks, idxMaks =
IPKTerbesar(arrMhs[:])     mhsMin, idxMin =
IPKTerkecil(arrMhs[:])
     fmt.Println("=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===")
fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan NIM %s, ditemukan
pada indeks ke-%d", mhsMaks.IPK, mhsMaks.nama, mhsMaks.nim, idxMaks)
fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan NIM %s, ditemukan
pada indeks ke-%d", mhsMin.IPK, mhsMin.nama, mhsMin.nim, idxMin)
}
```

Output :



```
DATA MAHASISWA INDEKS KE- 0
Masukkan nama : dwi
Masukkan NIM :100
Masukkan IPK : 3.5
=======================
DATA MAHASISWA INDEKS KE- 1
Masukkan nama : anwar
Masukkan NIM :23
Masukkan IPK : 2.4
=======================
DATA MAHASISWA INDEKS KE- 2
Masukkan nama : nandit
Masukkan NIM :1
Masukkan IPK : 2.1
=======================

DATA MAHASISWA INDEKS KE- 0
Nama :  dwi
NIM :  100
IPK :  3.5
=======================
DATA MAHASISWA INDEKS KE- 1
Nama :  anwar
NIM :  23
IPK :  2.4
=======================
DATA MAHASISWA INDEKS KE- 2
Nama :  nandit
NIM :  1
IPK :  2.1
=======================

=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===
IPK Terbesar : 3.50, atas nama dwi dengan NIM 100, dite
mukan pada indeks ke-0IPK Terkecil : 2.10, atas nama na
ndit dengan NIM 1, ditemukan pada indeks ke-2
```

Penjelasan :

Program dimulai dengan deklarasi tipe mahasiswa yang menyimpan tiga bidang: nama, nim, dan IPK. Kemudian didefinisikan arrMahasiswa sebagai array tetap berisi tiga mahasiswa, sehingga jumlah data yang diproses akan selalu tiga.

C. Unguided

1. Soal Unguided 1

Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

Masukan terdiri dari sekumpulan bilangan, yang mana bilangan pertama adalah bilangan bulat N yang menyatakan banyaknya anak kelinci yang akan ditimbang beratnya. Selanjutnya N bilangan riil berikutnya adalah berat dari anak kelinci yang akan dijual.

Keluaran terdiri dari dua buah bilangan riil yang menyatakan berat kelinci terkecil dan terbesar.

Jawaban:

```
package main
import "fmt"
 func main() {     var n
int     var berat
[1000]float64     var
min, max float64

fmt.Scan(&n)
     for i := 0; i < n;
i++ {
fmt.Scan(&berat[i])
    }      min =
berat[0]     max
= berat[0]
     for i := 1; i < n;
i++ {         if berat[i]
< min {             min =
berat[i]
        }         if
berat[i] > max {
max = berat[i]
        }
    }      fmt.Printf("%.2f %.2f\n",
min, max)
}
```

Output :

```
sar\Week 9> cd "Unguided"
PS C:\Users\LENOVO\Desktop\109082500182_Adib-Hanan-Kaut
sar\Week 9\Unguided> go run 1.go
4
1
2
3
4
1.00 4.00
PS C:\Users\LENOVO\Desktop\109082500182_Adib-Hanan-Kaut
sar\Week 9\Unguided>
```

Penjelasan :
Program ini bertujuan mencari berat terkecil dan terbesar dari N ekor kelinci dengan cara menyimpan seluruh data ke dalam array berkapasitas 1000 dan menerapkan algoritma pencarian nilai ekstrim dasar. Secara teknis, program menetapkan berat kelinci pertama sebagai nilai minimum dan maksimum sementara, lalu melakukan perulangan untuk membandingkan data tersebut dengan berat kelinci lainnya

guna memperbarui nilai ekstrim jika ditemukan data yang lebih kecil atau lebih besar.

2. Soal Unguided 2

Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Masukan terdiri dari dua baris, yang mana baris pertama terdiri dari dua bilangan bulat x dan y. Bilangan x menyatakan banyaknya ikan yang akan dijual, sedangkan y adalah banyaknya ikan yang akan dimasukan ke dalam wadah. Baris kedua terdiri dari sejumlah x bilangan riil yang menyatakan banyaknya ikan yang akan dijual.

Keluaran terdiri dari dua baris. Baris pertama adalah kumpulan bilangan riil yang menyatakan total berat ikan di setiap wadah (jumlah wadah tergantung pada nilai x dan y, urutan ikan yang dimasukan ke dalam wadah sesuai urutan pada masukan baris ke-2). Baris kedua adalah sebuah bilangan riil yang menyatakan berat rata-rata ikan di setiap wadah.

Jawaban :

```
package main
import "fmt"
 func main() {     var x, y
int     var beratIkan
[1000]float64
         fmt.Scan(&x, &y)
for i := 0; i < x; i++ {
fmt.Scan(&beratIkan[i])
    }      var totalSeluruhBerat
float64 = 0     var jumlahWadah int
= 0     var beratWadah float64 = 0
          for i := 0; i < x; i++ {
beratWadah += beratIkan[i]
totalSeluruhBerat += beratIkan[i]
                          if (i+1)%y == 0
|| i == x-1 {
fmt.Printf("%.2f ", beratWadah)
beratWadah = 0             jumlahWadah++
```

```
        }     }
fmt.Println()
          fmt.Printf("%.2f\n",
totalSeluruhBerat/float64(jumlahWadah)) }
```

Output :

```
PS C:\Users\LENOVO\Desktop\109082500182_Adib-Hanan-Kautsar\Week 9\
Unguided> go run 2.go
1
2
3
3.00
3.00
PS C:\Users\LENOVO\Desktop\109082500182_Adib-Hanan-Kautsar\Week 9\
Unguided> 4
```

Penjelasan :

Program ini berfokus pada pengelompokan data berat ikan ke dalam beberapa wadah berdasarkan kapasitas y yang ditentukan oleh pengguna, lalu menghitung total berat di setiap wadah secara berurutan. Selain menjumlahkan berat ikan per wadah menggunakan struktur perulangan, program ini juga menghitung nilai rata-rata berat seluruh wadah yang telah terisi sebagai informasi tambahan bagi penjual.

3. Soal Unguided 3

Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya.

Buatlah program dengan spesifikasi subprogram sebagai berikut:


```
type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita; bMin, bMax *float64) {
/* I.S. Terdefinisi array dinamis arrBerat
   Proses: Menghitung berat minimum dan maksimum dalam array
   F.S. Menampilkan berat minimum dan maksimum balita */
     ...
}

function rerata (arrBerat arrBalita) real {
/* menghitung dan mengembalikan rerata berat balita dalam array */
     ...
}
```

Perhatikan sesi interaksi pada contoh berikut ini (**teks bergaris bawah adalah input/read**)

```
Masukan banyak data berat balita : 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38 kg
```

Fakultas Informatika
School of Computing

Jawaban :

```
package main
import "fmt"
 type arrBalita
[100]float64
 func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax
*float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]
     for i := 1; i < n; i++ {
if *bMin > arrBerat[i] {
*bMin = arrBerat[i]
        }         if *bMax <
arrBerat[i] {
*bMax = arrBerat[i]
        }
    }
}  func rerata(arrBerat arrBalita, n int)
float64 {     var total float64 = 0     for i
:= 0; i < n; i++ {         total +=
arrBerat[i]
    }     return total /
float64(n)
}  func main() {
var n int     var data
arrBalita     var min,
max float64

    fmt.Print("Masukan banyak data berat balita: ")
fmt.Scan(&n)

    for i := 0; i < n; i++ {
fmt.Printf("Masukan berat balita ke-%d: ", i+1)
fmt.Scan(&data[i])
    }      hitungMinMax(data, n,
&min, &max)
         fmt.Printf("Berat balita minimum: %.2f kg\n", min)
fmt.Printf("Berat balita maksimum: %.2f kg\n", max)
fmt.Printf("Rerata berat balita: %.2f\n", rerata(data, n))
```

```
}
```

Output :

```
3.00
PS C:\Users\LENOVO\Desktop\109082500182_Adib-Hanan-Kautsar\Week 9\Un
guided> go run 3.go
Masukan banyak data berat balita: 11
Masukan berat balita ke-1: 15
Masukan berat balita ke-2: 14
Masukan berat balita ke-3: 11
Masukan berat balita ke-4: 10
Masukan berat balita ke-5: 16
Masukan berat balita ke-6: 10
Masukan berat balita ke-7: 11
Masukan berat balita ke-8: 1
Masukan berat balita ke-9: 00
Masukan berat balita ke-10: 10
Masukan berat balita ke-11: 20
Berat balita minimum: 0.00 kg
Berat balita maksimum: 20.00 kg
Rerata berat balita: 10.73
PS C:\Users\LENOVO\Desktop\109082500182_Adib-Hanan-Kautsar\Week 9\Un
guided>
```

Penjelasan :
Program ini mengimplementasikan pencarian nilai ekstrim melalui pendekatan modular dengan membagi tugas ke dalam subprogram, yaitu prosedur hitung Min Max dan fungsi rerata. Prosedur hitung Min Max menggunakan parameter *pointer* agar dapat menghasilkan nilai minimum dan maksimum balita secara bersamaan dalam satu proses, sementara fungsi rerata bekerja secara spesifik untuk menjumlahkan seluruh data berat dalam *array* dan mengembalikan hasil pembagian rata-ratanya.

D. Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa Pencarian nilai maksimum dan minimum dilakukan dengan membandingkan seluruh elemen array secara sekuensial, dimulai dengan menetapkan elemen pertama sebagai acuan awal. Pencarian pada tipe data terstruktur (struct) lebih efektif jika dilakukan dengan mencari dan menyimpan indeksnya, sehingga seluruh informasi terkait objek tersebut dapat diambil dengan mudah. Penerapan modular dengan menggunakan fungsi dan prosedur (terutama dengan parameter pointer) memudahkan pengelolaan data dan memungkinkan program untuk menghasilkan beberapa informasi sekaligus (seperti min, max, dan rerata) secara lebih terorganisir.